



**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ»
(национальный исследовательский университет)**

УТВЕРЖДАЮ

Заведующий кафедрой 806

(№ каф.)

_____ Крылов С. С.

(подпись)

(инициалы, фамилия)

_____ 2026 г.

**ОТЧЕТ
О НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ**

по теме:

**Платформы видеоаналитики
реального времени для мониторинга и событийного
анализа в системах охранного видеонаблюдения.**

Научный руководитель _____

Гаврилов Константин

Юрьевич

доктор технических наук,

профессор

Исполнитель _____

Нурмухамедов А. Б.

Москва 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 ПОСТАНОВКА ЗАДАЧИ	5
1.1 Проблема предметной области	5
1.2 Ключевые требования	5
1.3 Принятые ограничения	6
1.4 Цель и ожидаемый результат	6
2 АРХИТЕКТУРА И АЛГОРИТМИЧЕСКИЙ ПОДХОД	8
2.1 Логика архитектуры	8
2.2 Алгоритмический конвейер	9
2.2.1 Детекция	9
2.2.2 Сопровождение	10
2.2.3 Событийный анализ	10
2.3 Надежная доставка событий	11
2.4 Наблюдаемость и управление качеством	14
2.5 Архитектурные драйверы и границы системы	14
2.6 Поток данных и контракты взаимодействия	15
2.7 Модель данных и жизненный цикл события	16
2.8 Масштабирование и расчет эксплуатационной емкости	18
2.9 Архитектура отказоустойчивости и деградационные режимы	19
2.10 Безопасность архитектурного контура	20
2.11 Архитектурные выводы для промышленного перехода	20
2.12 Операционный регламент архитектурного контура	21
3 РЕАЛИЗАЦИЯ И ОСНОВНЫЕ РЕЗУЛЬТАТЫ	23
3.1 Технологическая реализация	23
3.2 Операторский интерфейс	23
3.3 Результаты интеграционной проверки	25
3.4 Производительность и эксплуатационный смысл метрик	26
3.5 Проверка надежности в отказных режимах	27
3.6 Ограничения текущей апробации	27

3.7	Практические рекомендации	28
3.8	Приоритетные направления развития	28
ЗАКЛЮЧЕНИЕ		29
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ		30

ВВЕДЕНИЕ

Системы охранного видеонаблюдения работают в условиях постоянного роста числа камер и объема видеоданных. Ручной мониторинг при такой нагрузке приводит к пропускам событий и увеличению времени реакции. Поэтому основная практическая задача заключается в переходе к автоматическому анализу видеопотоков в режиме, близком к реальному времени.

В отчете рассматривается магистерская работа по теме «Платформы видеоаналитики реального времени для мониторинга и событийного анализа в системах охранного видеонаблюдения». В рамках работы разработана и апробирована платформа, которая выполняет обнаружение объектов, сопровождение во времени, событийную интерпретацию и надежную доставку сведений о событиях внешним потребителям.

Актуальность определяется необходимостью одновременно решать две группы задач: алгоритмические (обнаружение и сопровождение объектов) и инженерные (устойчивый интеграционный контур, безопасность, наблюдаемость) [1—3].

Цель работы состоит в разработке и экспериментальной проверке на исследовательско-экспериментальном стенде платформы видеоаналитики реального времени для мониторинга охраняемых зон и событийного анализа, обеспечивающей обнаружение и сопровождение объектов, формирование интерпретируемых событий и надежную доставку сведений о событиях внешним потребителям.

Для достижения цели необходимо решить следующие задачи:

- 1) проанализировать предметную область охранного видеонаблюдения, существующие подходы к построению платформ видеоаналитики и сформировать функциональные и нефункциональные требования к разрабатываемому решению;
- 2) обосновать выбор методов обнаружения объектов, сопровождения объектов и правил событийного анализа для целевых сценариев видеонаблюдения, включая демонстрационный сценарий обнаружения беспилотных летательных аппаратов;
- 3) разработать модель, архитектуру и алгоритмический конвейер плат-

формы, объединяющие прием видеопотока, обнаружение, сопровождение, формирование событий и асинхронную доставку сведений о событиях внешним потребителям;

- 4) реализовать программную платформу с компонентами управления, аналитической обработки, интеграционной доставки, разграничения доступа, наблюдаемости и воспроизводимого развертывания;
- 5) провести стендовую апробацию платформы по интеграционным, отказным, ролевым и демонстрационным видеосценариям с фиксацией границ доказанности полученных результатов;
- 6) определить ограничения текущей реализации, условия ее применимости и направления дальнейшего развития для перехода от исследовательско-экспериментального стенда к промышленному применению.

Объект исследования - процессы автоматизированного мониторинга видеопотоков в системах охранного видеонаблюдения. Предмет исследования - методы, модели и архитектурные решения платформ видеоаналитики реального времени, обеспечивающие обнаружение и сопровождение объектов, формирование событий и надежную доставку сведений о событиях при ограничениях по задержке обработки, отказоустойчивости и безопасности доступа.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Проблема предметной области

В охранном видеонаблюдении полезным результатом является не детекция как таковая, а своевременно сформированный факт события. Для этого система должна выполнять четыре связанные функции:

- 1) принимать видеопоток и поддерживать стабильную обработку кадров;
- 2) обнаруживать и классифицировать объекты;
- 3) сопровождать их во времени с сохранением идентичности;
- 4) интерпретировать траектории в терминах событий и надежно доставлять сведения о событиях.

Пусть для камеры i поток кадров обозначен как $\{x_t^i\}$. Тогда целевой конвейер задается последовательностью преобразований:

$$\mathcal{V} \rightarrow \mathcal{D} \rightarrow \mathcal{T} \rightarrow \mathcal{E} \rightarrow \text{event_outbox}, \quad (1)$$

где $\mathcal{D}$ — детекции;

$\mathcal{T}$ — треки;

$\mathcal{E}$ — события.

Графически сквозной маршрут показан на рисунке 1.1. Каждый узел представляет дискретный этап преобразования данных, переход между этапами реализован через явный программный контракт.

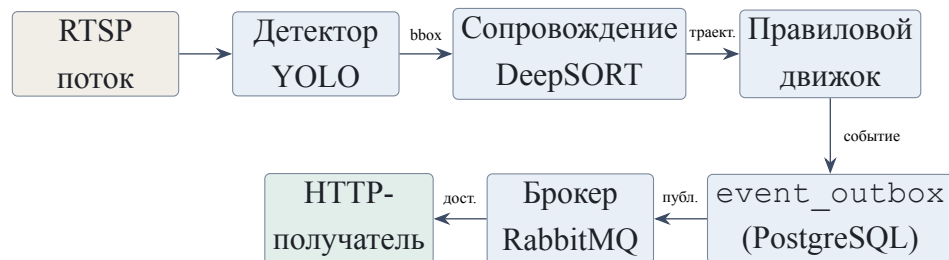


Рисунок 1.1 - Сквозной конвейер обработки видео и доставки событий

1.2 Ключевые требования

Требования зафиксированы в связке «требование - проверка».

Таблица 1.1 - Ключевые требования к платформе

Требование	Практический критерий
Работа в режиме реального времени	Ограниченный p95 object->event, отсутствие неограниченного роста очередей
Надежность интеграции	Сохранение событий в таблице исходящих сообщений при отказах, успешная обработка накопленной очереди после восстановления
Интерпретируемость событий	Явное правило формирования (zone_enter, line_cross, dwell_time) и трассируемый dedup_key
Безопасность доступа	Корректный сценарий JWT-аутентификации, соблюдение RBAC-ограничений для ролей admin/operator/viewer
Воспроизводимость эксплуатации	Единый compose-стенд, наблюдаемость через /health и /metrics

1.3 Принятые ограничения

Чтобы получить измеримый и воспроизводимый результат в магистерском проекте, осознанно приняты ограничения:

- базовый охранный профиль ориентирован на классы person и vehicle; БПЛА вынесены в отдельный демонстрационный профиль с классами bird и drone;
- камерно-изолированное сопровождение без межкамерной повторной идентификации (ReID);
- односерверный контур развертывания;
- ограниченный, но интерпретируемый набор событийных правил.

1.4 Цель и ожидаемый результат

Цель работы состоит в разработке и экспериментальной проверке на исследовательско-экспериментальном стенде платформы видеоаналитики реального времени для мониторинга охраняемых зон и событийного анализа, обеспечивающей обнаружение и сопровождение объектов, формирова-

ние интерпретируемых событий и надежную доставку сведений о событиях внешним потребителям.

Ожидаемый результат - воспроизводимый исследовательско-экспериментальный стенд платформы, в котором подтверждены:

- 1) корректность интеграционной доставки синтетически зафиксированных событий `zone_enter`, `line_cross` и `dwelling_time`;
- 2) связность демонстрационного видеоконтра БПЛА от обученной модели до траекторий, событий и операторского интерфейса;
- 3) устойчивость контуров доставки при инфраструктурных сбоях;
- 4) работоспособность ролевой модели доступа;
- 5) наблюдаемость состояния платформы по ключевым метрикам.

2 АРХИТЕКТУРА И АЛГОРИТМИЧЕСКИЙ ПОДХОД

2.1 Логика архитектуры

Архитектура построена как набор специализированных сервисов с четким разделением ответственности:

- `api` - управление конфигурацией, доступом и журналом событий;
- `analytics-worker` - обработка видеопотоков и генерация событий;
- `integration-worker` - доставка событий через брокер и HTTP-получатель;
- `ui` - операторские сценарии;
- инфраструктура: PostgreSQL, RabbitMQ, Prometheus, Grafana.

Контекстный уровень C4 (рисунок 2.1) отражает положение платформы относительно внешних участников: оператор и администратор работают через пользовательский интерфейс и API, IP-камеры подают RTSP-поток, а внешние корпоративные системы получают события через HTTP-получатель.



Рисунок 2.1 - C4: контекстный уровень платформы и внешние участники

Уровень контейнеров (рисунок 2.2) детализирует внутреннюю композицию: контур управления (API, пользовательский интерфейс, БД), контур обработки данных (`analytics-worker`, `integration-worker`, RabbitMQ) и инфраструктура наблюдаемости.

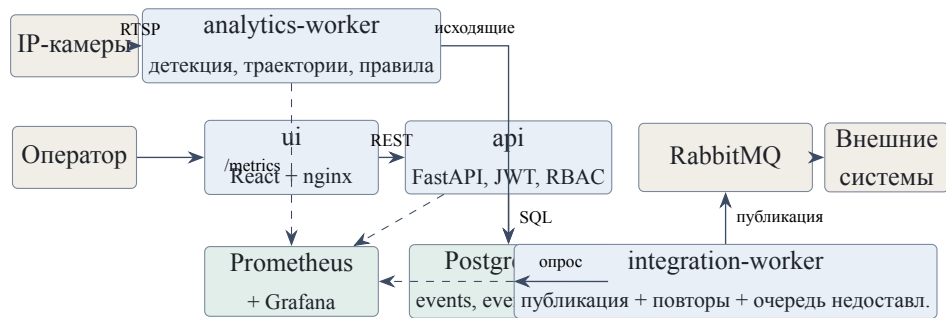


Рисунок 2.2 - С4: контейнерный уровень и композиция сервисов платформы

2.2 Алгоритмический конвейер

В основе аналитического слоя - связка детектор + модуль сопровождения + правилевой движок.

2.2.1 Детекция

В качестве базового метода выбран одноэтапный детектор семейства YOLO как компромисс между скоростью и точностью в потоковой обработке [1]. После инференса применяются порог достоверности и подавление дубликатов по IoU.

Пример отклика демонстрационного БПЛА-профиля показан на рисунке 2.3: для классов `bird` и `drone` формируются прямоугольные рамки с оценкой достоверности, которые далее передаются в модуль сопровождения.



Рисунок 2.3 - Пример отклика детектора в демонстрационном БПЛА-сценарии

2.2.2 Сопровождение

Для устойчивого сохранения идентичности используется подход сопровождения по результатам обнаружения, реализованный в логике DeepSORT: он сочетает кинематический прогноз и признаки внешнего вида [2]. Это снижает число переключений идентификаторов при частичных перекрытиях и временных пропусках детекций.

2.2.3 Событийный анализ

Событие формируется правилом, заданным как пространственно-временной предикат над траекторией:

$$e_r = P_r(\tau_j, \mathcal{Z}, t), \quad (1)$$

где P_r — предикат правила одного из типов: `zone_enter`, `line_cross`, `dwell_time`.

Реализация правил локализована в классе `RuleEngine`: для каждого трека хранится последняя позиция и время входа в зону, а проверка выполняется по геометрическим предикатам. Листинг 2.1 приведен из файла `services/analytics-worker/app/rule_engine.py`.

Листинг 2.1: Фрагмент правил `line_cross` и `dwell_time`

```
1 if rule_type == "line_cross":
2     line_points = (params.get("line") or [])
3     if len(line_points) == 2 and st.last_point is not None:
4         p1 = (float(line_points[0][0]), float(line_points[0][1]))
5         p2 = (float(line_points[1][0]), float(line_points[1][1]))
6         line_key = f"line_cross:{rule['rule_id']}:{track_id}"
7         if segments_intersect(st.last_point, current_point, p1, p2) \
8             and line_key not in st.fired_keys:
9             st.fired_keys.add(line_key)
10            dedup = f"{self.camera_id}:{rule['rule_id']}:{track_id}:line_cross"
11            generated.append(self._build_event(
12                track=track, rule=rule,
13                event_type="line_cross",
14                dedup_key=dedup, zone_id=zone_id,
15            ))
16
17 if rule_type == "dwell_time" and zone:
18     threshold = int(params.get("seconds", 5))
19     polygon = (zone.get("geometry") or {}).get("points", [])
20     inside = point_in_polygon(current_point, polygon)
21     if inside:
22         start_ts = st.zone_entered_at.setdefault(zone_id, ts)
23         dwell_key = f"{rule['rule_id']}:{zone_id}:{track_id}:{threshold}"
24         if (ts - start_ts).total_seconds() >= threshold \
25             and dwell_key not in st.fired_keys:
26             st.fired_keys.add(dwell_key)
27     ...
```

Здесь `fired_keys` обеспечивает идемпотентность срабатывания: повторный вход траектории в ту же зону в рамках одного активного эпизода не приводит к дублированию события. Поле `dedup_key` включает идентификаторы камеры, правила и траектории, что позволяет внешнему потребителю безопасно повторно обрабатывать сообщения.

2.3 Надежная доставка событий

Ключевое инженерное решение - транзакционная таблица исходящих сообщений [3; 4]. Событие и запись исходящего сообщения фиксируются в одной транзакции БД, после чего публикация в брокер выполняется асин-

хронно:

`commit(событие, исходящее сообщение)` $\Rightarrow$ событие не теряется. (2)

При временной недоступности брокера событие остается в таблице исходящих сообщений и публикуется после восстановления внешнего контура. Для интеграционного слоя используется политика повторов с ограниченной задержкой между попытками и маршрутизацией в очередь недоставленных сообщений после исчерпания попыток. Это дает контролируемое поведение в отказных режимах и прозрачную диагностику.

Жизненный цикл сообщения по контурам ответственности показан на рисунке 2.4: фиксация события и записи исходящего сообщения выполняется в одной транзакции, а доставка продолжается асинхронно даже при кратковременной недоступности RabbitMQ или HTTP-получателя.

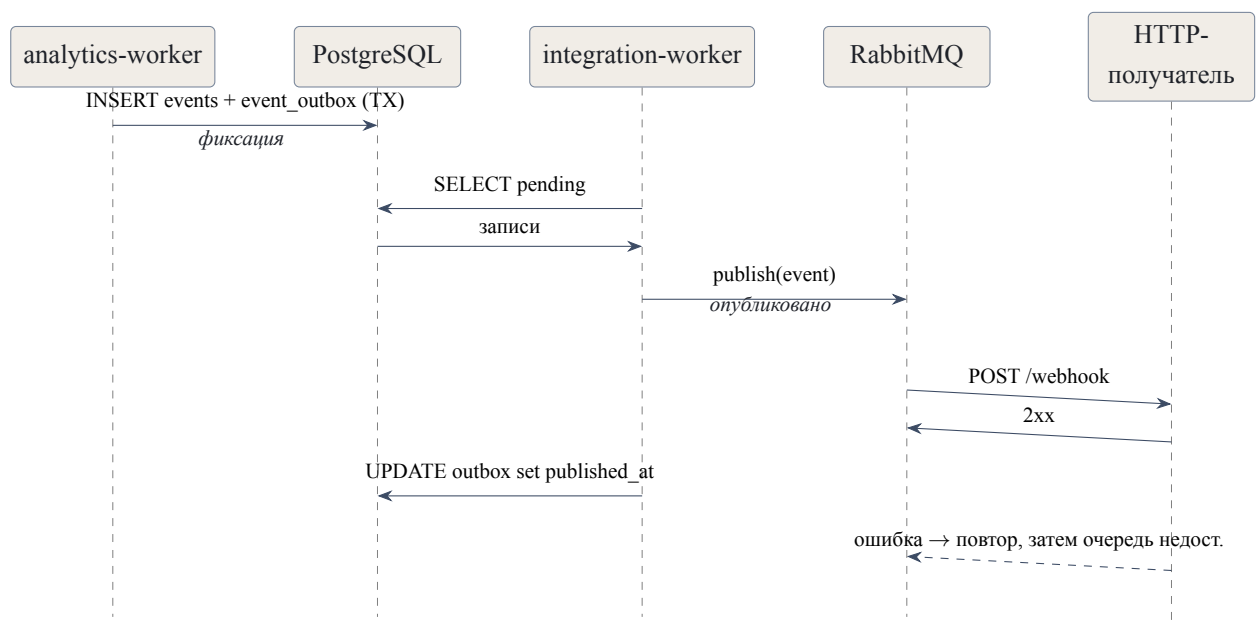


Рисунок 2.4 - Диаграмма последовательности доставки события через таблицу исходящих сообщений

Реализация цикла публикации в `integration-worker` выделяет три явные позиции отказа: соединение с брокером, публикацию сообщения и подтверждение записи исходящего сообщения. На каждом из них предусмотрена реакция, отделяющая повторяемые ошибки от конечных. Листинг 2.2 приведен из `services/integration-worker/app/main.py`.

Листинг 2.2: Цикл публикации исходящих сообщений с ограниченными повторами

```
1 candidates = fetch_publish_candidates(limit=100)
2 for row in candidates:
3     outbox_id = row["outbox_id"]
4     retry_count = int(row.get("retry_count", 0))
5     payload = row["payload"]
6     event_type = payload.get("event_type", "unknown")
7     try:
8         broker.publish_event(payload, event_type)
9         mark_published(outbox_id)
10        PUBLISH_SUCCESS.inc()
11    except Exception as exc:
12        retry_count += 1
13        terminal = retry_count >= settings.max_retry
14        mark_retry(outbox_id, retry_count, terminal=terminal)
15        PUBLISH_ERROR.inc()
16        if not terminal:
17            OUTBOX_RETRIES.inc()
18            # ошибка брокера: переподключаемся и продолжаем со следующего пакета
19            reconnect_required = True
20        break
```

Логика обработчика сообщения (доставка HTTP-получателю) дополняется экспоненциальной задержкой между повторами и переходом в очередь недоставленных сообщений после исчерпания попыток (листинг 2.3).

Листинг 2.3: Доставка HTTP-получателю с повтором и переходом в очередь недоставленных сообщений

```
1 success, error_text = _deliver_webhook(payload)
2 if success:
3     WEBHOOK_SUCCESS.inc()
4     log_delivery_attempt(event_id, consumer_name, "success", None)
5     channel.basic_ack(delivery_tag=method.delivery_tag)
6     return
7
8 WEBHOOK_FAILURE.inc()
9 log_delivery_attempt(event_id, consumer_name, "failed", error_text)
10 next_attempt = attempt + 1
11 if next_attempt >= settings.max_retry:
12     broker.publish_dlq(payload, reason=error_text or "max_retries")
13     DLQ_PUBLISH.inc()
14     channel.basic_ack(delivery_tag=method.delivery_tag)
15     return
16
17 time.sleep(min(30, 2 ** next_attempt)) # экспоненциальная задержка
18 broker.publish_event(payload, payload.get("event_type", "unknown"),
19                      headers={"delivery_attempt": next_attempt})
```

2.4 Наблюдаемость и управление качеством

Платформа проектировалась как наблюдаемая система. В каждом сервисе реализованы:

- `/health/live`, `/health/ready`;
- `/metrics` для Prometheus;
- журналирование действий и ошибок по сервисным сценариям.

Ключевые эксплуатационные метрики:

- 1) p95 задержка `object->event`;
- 2) доля отброшенных кадров;
- 3) размер накопленной очереди исходящих сообщений и число повторов;
- 4) число сообщений в очереди недоставленных;
- 5) успешность доставки HTTP-получателю.

2.5 Архитектурные драйверы и границы системы

Архитектура второй итерации отчета уточнена через набор архитектурных драйверов, которые напрямую влияют на структуру компонентов:

- актуальность события важнее обработки каждого кадра без исключения;
- отказ внешней интеграции не должен блокировать аналитический контур;
- замена детектора или модуля сопровождения должна быть локальным изменением;
- эксплуатационная диагностика должна выполняться без доступа к исходному коду;
- конфигурация камер и правил должна изменяться без остановки всей платформы.

По этой причине система разделена на два функциональных слоя:

- контур управления - API управления, конфигурация, аудит, ролевой доступ;
- контур обработки данных - захват потока, инференс, сопровождение объектов, формирование и доставка событий.

Практический эффект такого разделения состоит в том, что при про-

блемах в контуре управления допустима продолженная обработка видео по уже загруженным правилам, а при временной перегрузке контура обработки данных административные функции остаются доступными.

2.6 Поток данных и контракты взаимодействия

Сквозной маршрут в эксплуатационном режиме фиксируется как последовательность этапов:

- 1) прием RTSP-потока и декодирование;
- 2) нормализация таймштампов и отбор актуального кадра;
- 3) инференс детектора и фильтрация дубликатов;
- 4) обновление состояния траекторий по камере;
- 5) применение правил к траекториям;
- 6) транзакционная фиксация `events` и `event_outbox`;
- 7) асинхронная публикация в брокер;
- 8) доставка во внешний HTTP-получатель или API.

Для устранения неоднозначностей в интеграции закреплён минимальный контракт события (таблица 2.1).

Таблица 2.1 - Минимальный контракт доменного события

Поле	Тип	Назначение
event_id	UUID	Уникальный идентификатор события
event_type	string	Тип события: zone_enter, line_cross, dwell_time
camera_id	UUID/string	Камера-источник события
track_id	integer	Идентификатор траектории внутри камеры
event_time	timestamp	Момент выполнения правила
rule_id	UUID/string	Правило, сгенерировавшее событие
dedup_key	string	Ключ дедупликации при повторной доставке
payload	JSON	Расширенные данные: зона, линия, оценка достоверности, bbox

Внутренний API проектируется по принципу явных границ: синхронные HTTP-вызовы используются только для управления и чтения состояния, а доменные события и интеграция выносятся в асинхронный контур.

2.7 Модель данных и жизненный цикл события

Событийная запись проходит набор дискретных состояний:

- 1) created - событие сформировано правилом;
- 2) persisted - запись сохранена в events и event_outbox;
- 3) published - сообщение подтверждено брокером;
- 4) delivered - внешний получатель подтвердил прием;
- 5) dead_letter - попытки исчерпаны, запись перенаправлена в очередь недоставленных.

ER-диаграмма ключевых таблиц предметной области (рисунок 2.5) показывает связи между конфигурацией (камеры, зоны, правила), оперативным состоянием (траектории) и фактологическим слоем (события и исхо-

дящие сообщения). Дедупликация событий обеспечивается уникальностью `dedup_key` в таблице `events`, а `event_outbox` разделяет фиксацию факта и публикацию сообщения.

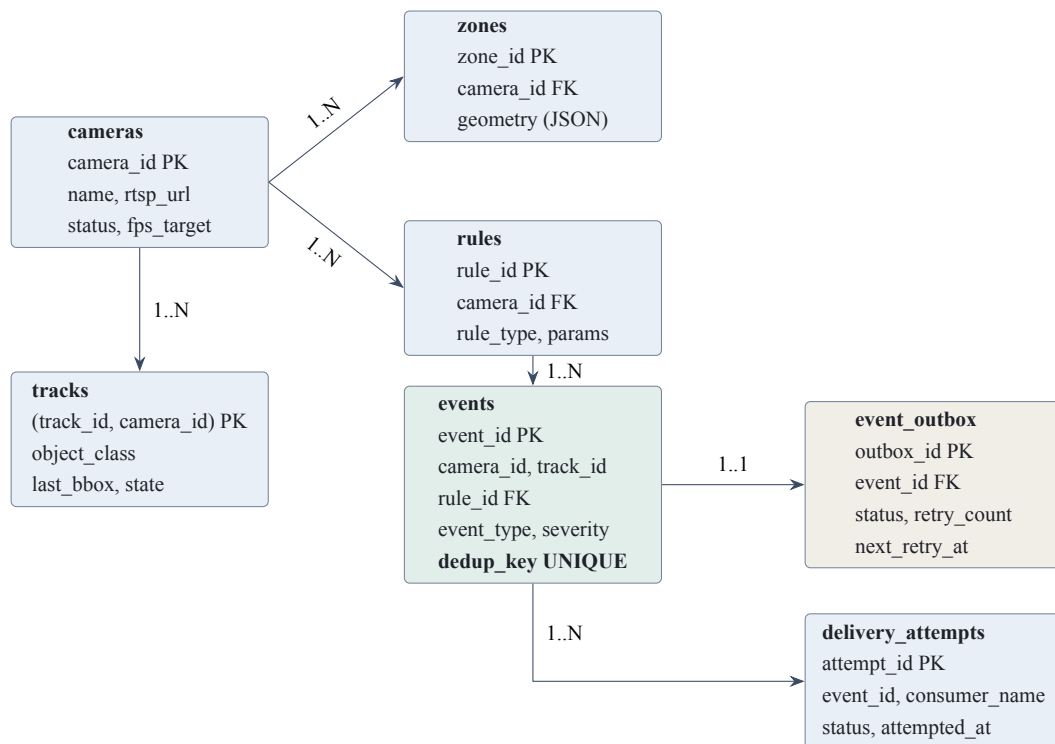


Рисунок 2.5 - ER-диаграмма ключевых таблиц предметной области

Декларация ORM для событий и исходящих сообщений приведена в листинге 2.4 (фрагмент `services/api/app/models.py`). Уникальность `dedup_key` вынесена на уровень схемы, что исключает повторную вставку даже при ошибочной попытке записи.

Листинг 2.4: Декларация моделей `Event` и `EventOutbox`

```

1 class Event(Base):
2     __tablename__ = "events"
3     __table_args__ = (UniqueConstraint("dedup_key", name="uq_events_dedup_key"),)
4
5     event_id: Mapped[str] = mapped_column(String(36), primary_key=True, default=lambda: str(
6         ↪ uuid4()))
7     camera_id: Mapped[str] = mapped_column(ForeignKey("cameras.camera_id", ondelete="CASCADE")
8         ↪ , nullable=False, index=True)
9     track_id: Mapped[str] = mapped_column(String(64), nullable=False, index=True)
10    rule_id: Mapped[str] = mapped_column(ForeignKey("rules.rule_id", ondelete="SET NULL"),
11        ↪ nullable=True, index=True)
12    event_type: Mapped[str] = mapped_column(String(32), nullable=False, index=True)
13    severity: Mapped[str] = mapped_column(String(16), nullable=False)
14    occurred_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False,
15        ↪ index=True)
16    confidence: Mapped[float] = mapped_column(Float, nullable=False)
17    dedup_key: Mapped[str] = mapped_column(String(255), nullable=False)
18    attributes: Mapped[dict] = mapped_column(JSON, nullable=False, default=dict)

```

```

15
16
17 class EventOutbox(Base):
18     __tablename__ = "event_outbox"
19     outbox_id: Mapped[str] = mapped_column(String(36), primary_key=True, default=lambda: str(
20         ↪ uuid4()))
21     event_id: Mapped[str] = mapped_column(ForeignKey("events.event_id", ondelete="CASCADE"),
22         ↪ nullable=False, index=True)
23     payload: Mapped[dict] = mapped_column(JSON, nullable=False)
24     status: Mapped[str] = mapped_column(String(16), nullable=False, default="pending", index=
25         ↪ True)
26     retry_count: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
27     next_retry_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True), nullable=
28         ↪ True, index=True)
29     published_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True), nullable=
30         ↪ True)

```

Формально целостность состояния публикации задается условием:

$$N_{\text{created}} = N_{\text{delivered}} + N_{\text{outbox_pending}} + N_{\text{dead_letter}}, \quad (3)$$

где N_{created} — число созданных событий за интервал наблюдения;

$N_{\text{delivered}}$ — число успешно доставленных событий;

$N_{\text{outbox_pending}}$ — число ожидающих публикации/доставки событий;

$N_{\text{dead_letter}}$ — число событий, ушедших в очередь недоставленных после исчерпания повторов.

Условие используется как проверка консистентности эксплуатационного контура: рост `outbox_pending` или `dead_letter` при стабильной входной нагрузке трактуется как признак деградации интеграционного слоя.

2.8 Масштабирование и расчет эксплуатационной емкости

Для практической оценки применена модель пропускной способности по камерам. Пусть λ_i - входная интенсивность кадров для камеры i , μ_i - интенсивность обработки горячего пути, δ_i - интенсивность контролируемого отбрасывания устаревших кадров. Тогда условие стабилизации очереди имеет вид:

$$\lambda_i \leq \mu_i + \delta_i. \quad (4)$$

Здесь δ_i не является дефектом алгоритма, а архитектурным инструментом сохранения актуальности результата при пиковых нагрузках. Для всей

платформы агрегированное условие записывается как

$$\sum_{i=1}^K \lambda_i \leq \sum_{i=1}^K (\mu_i + \delta_i), \quad (5)$$

где K — число одновременно обслуживаемых камер.

Сквозная задержка события декомпозируется:

$$L_{\text{object} \rightarrow \text{event}} = L_{\text{decode}} + L_{\text{infer}} + L_{\text{track}} + L_{\text{rule}} + L_{\text{outbox}} + L_{\text{publish}}. \quad (6)$$

Такая декомпозиция позволяет локализовать узкое место: например, рост L_{infer} указывает на необходимость оптимизации модели и контура исполнения, а рост L_{publish} - на проблемы брокера или внешнего получателя.

2.9 Архитектура отказоустойчивости и деградационные режимы

Отказоустойчивость реализована через сочетание изоляции и отложенной доставки:

- изоляция камеры в отдельном рабочем контуре предотвращает каскадный отказ;
- фиксация событий через транзакционную таблицу исходящих сообщений исключает потерю на границе БД/брокера;
- повторы с увеличением задержки ограничивают нагрузку при недоступности внешнего HTTP-получателя;
- очередь недоставленных сообщений обеспечивает контролируемый «последний рубеж» и аудит недоставленных сообщений.

При кратковременной недоступности RabbitMQ система продолжает запись доменных событий в БД и накапливает очередь исходящих сообщений. После восстановления брокера публикация завершается без повторной генерации события на уровне аналитического ядра. При недоступности HTTP-получателя интеграционный сервис повторяет доставку и переводит неуспешные сообщения в очередь недоставленных сообщений.

Конечный автомат состояний записи исходящего сообщения показан на рисунке 2.6.

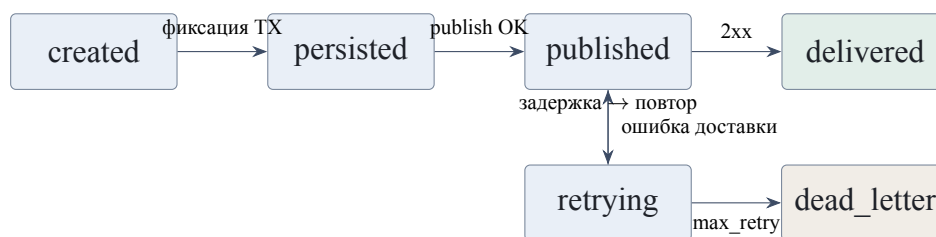


Рисунок 2.6 - Конечный автомат состояний записи исходящего сообщения

Практически это означает, что отказ внешнего контура снижает своевременность, но не разрушает достоверность истории событий в системе.

2.10 Безопасность архитектурного контура

Безопасность рассматривается как часть архитектуры, а не отдельный модуль. Применяются следующие решения:

- JWT-аутентификация для API-входа;
- RBAC-модель `admin/operator/viewer` для ограничения действий;
- аудит административных изменений (камеры, зоны, правила);
- валидация адресов HTTP-получателей и ограничение исходящих интеграций;
- разделение сетевых политик для служебных и пользовательских интерфейсов.

Для эксплуатационной приемки безопасности закреплены контрольные сценарии:

- 1) пользователь без роли `admin` не может изменять правила;
- 2) токен с истекшим сроком действия отклоняется на `gateway/API`;
- 3) чтение журнала событий доступно ролям, определенным политикой;
- 4) попытка обращения к внутренним маршрутам без авторизации фиксируется в аудите.

2.11 Архитектурные выводы для промышленного перехода

Дополнение архитектурной главы позволяет формализовать границы перехода к промышленному внедрению. Наиболее значимые условия такого перехода:

- сохранение контрактов доменных событий при доработке алгоритмического ядра;
- перенос тяжёлых компонентов инференса на профиль аппаратного ускорения без изменения бизнес-правил;
- поэтапное вынесение сервисов в распределенную схему только после подтверждения узких мест;
- сопровождение каждого архитектурного изменения эксплуатационными метриками и отказными тестами.

Именно эта последовательность обеспечивает управляемое масштабирование: сначала стабилизация контрактов и наблюдаемости, затем увеличение вычислительной емкости и только после этого усложнение топологии развертывания.

2.12 Операционный регламент архитектурного контура

Для практического применения архитектура должна сопровождаться регулярным техническим регламентом. В рамках короткого отчета зафиксирован минимальный набор процедур (таблица 2.2), который позволяет поддерживать устойчивость платформы между релизами.

Таблица 2.2 - Регламент контроля архитектурных свойств

Периодичность	Проверка	Критерий реакции
Каждый релиз	Сквозная цепочка detect->event->delivery	При нарушении сквозного сценария релиз не допускается в эксплуатацию
Ежедневно	размер накопленной очереди исходящих сообщений, очередь недоставленных, профиль повторов	При устойчивом росте запускается разбор инцидента и ограничение входной нагрузки
Еженедельно	регрессионные сценарии RBAC/JWT	При расхождении политик доступа выполняется откат конфигурации безопасности
Еженедельно	Проверка деградации p95 object->event	При ухудшении метрики инициируется анализ горячего пути и профилирование инференса
По инциденту	Сверка целостности событийного журнала и таблицы исходящих сообщений	Несоответствие состояния трактуется как критический дефект интеграционного контура

Введение такого регламента упрощает переход от исследовательского режима к эксплуатационному: архитектурные свойства становятся измеряемыми, а решения о доработках принимаются по наблюдаемым признакам, а не по субъективной оценке.

3 РЕАЛИЗАЦИЯ И ОСНОВНЫЕ РЕЗУЛЬТАТЫ

3.1 Технологическая реализация

Платформа реализована как набор изолированных микросервисов в одном репозитории. Основные технологии: FastAPI, PostgreSQL, RabbitMQ, Docker Compose, Prometheus и Grafana [4—9].

На уровне API реализована JWT-аутентификация с разграничением прав по ролям `admin`, `operator`, `viewer`.

Контракт доменного события формализован в JSON Schema (`contracts/event.schema.v1.json`). Сокращенный фрагмент приведен в листинге 3.1.

Листинг 3.1: Фрагмент схемы доменного события (JSON Schema)

```
1 {
2   "$schema": "https://json-schema.org/draft/2020-12/schema",
3   "type": "object",
4   "required": ["event_id", "event_type", "camera_id",
5               "track_id", "occurred_at", "dedup_key"],
6   "properties": {
7     "event_id": {"type": "string", "format": "uuid"},
8     "event_type": {"enum": ["zone_enter", "line_cross", "dwell_time"]},
9     "camera_id": {"type": "string"},
10    "track_id": {"type": "string"},
11    "rule_id": {"type": "string"},
12    "occurred_at": {"type": "string", "format": "date-time"},
13    "severity": {"enum": ["low", "medium", "high"]},
14    "confidence": {"type": "number", "minimum": 0, "maximum": 1},
15    "dedup_key": {"type": "string"},
16    "attributes": {"type": "object"}
17  }
18 }
```

3.2 Операторский интерфейс

Операторский интерфейс реализован как одностраничное приложение и закрывает три ключевых сценария: общий мониторинг (рисунок 3.1), просмотр статуса камер и зон (рисунок 3.2), а также журнал событий с фильтрацией по правилам и тяжести (рисунок 3.3).

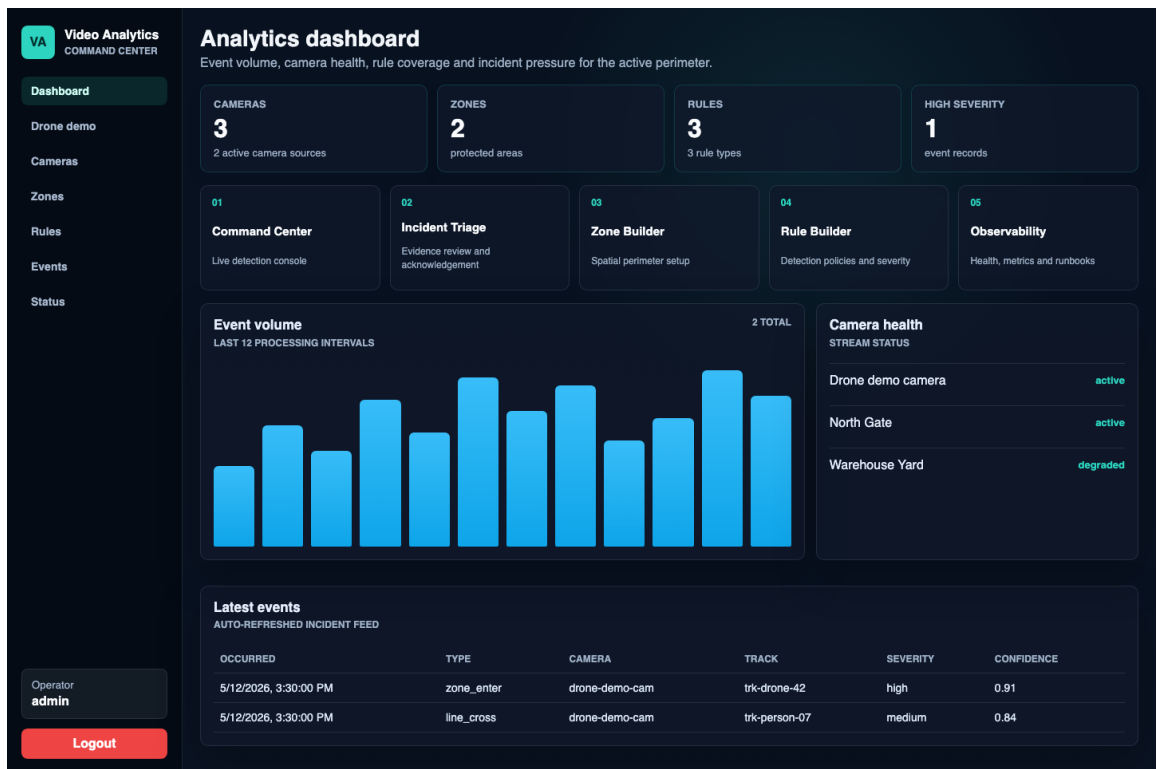


Рисунок 3.1 - Сводная панель оператора: состояние камер, активные правила и поток событий

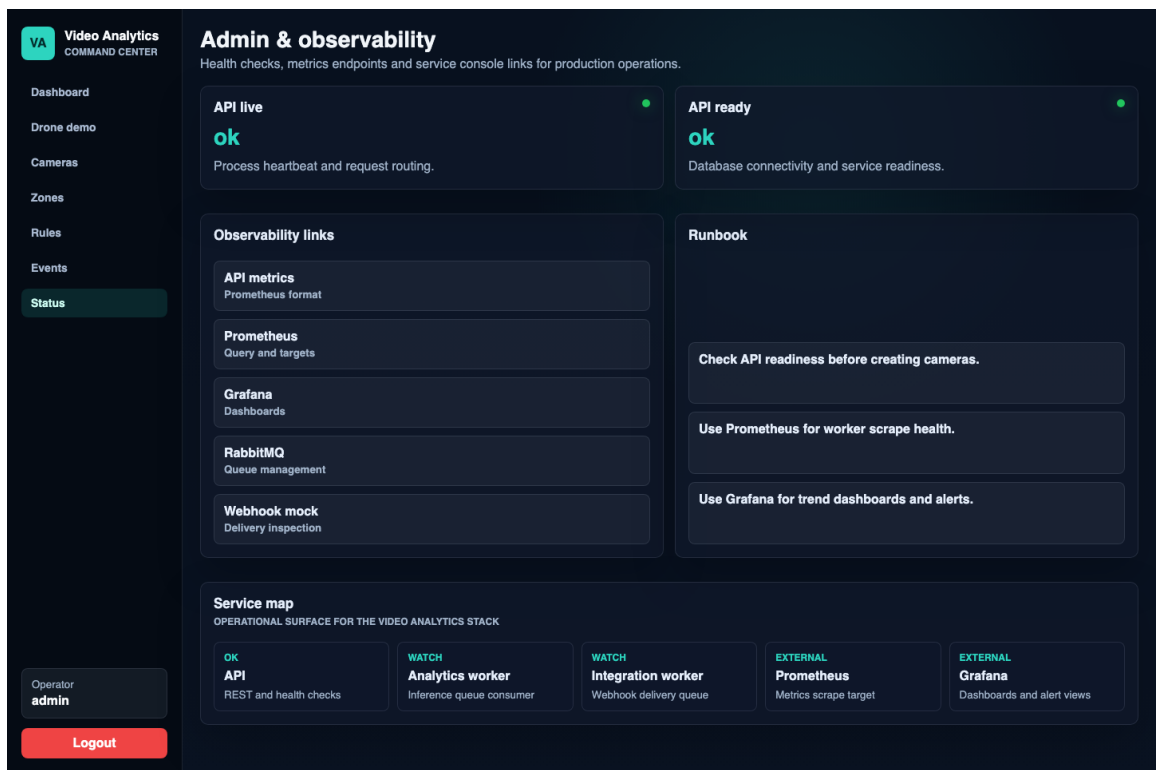


Рисунок 3.2 - Состояние камер и зон в реальном времени

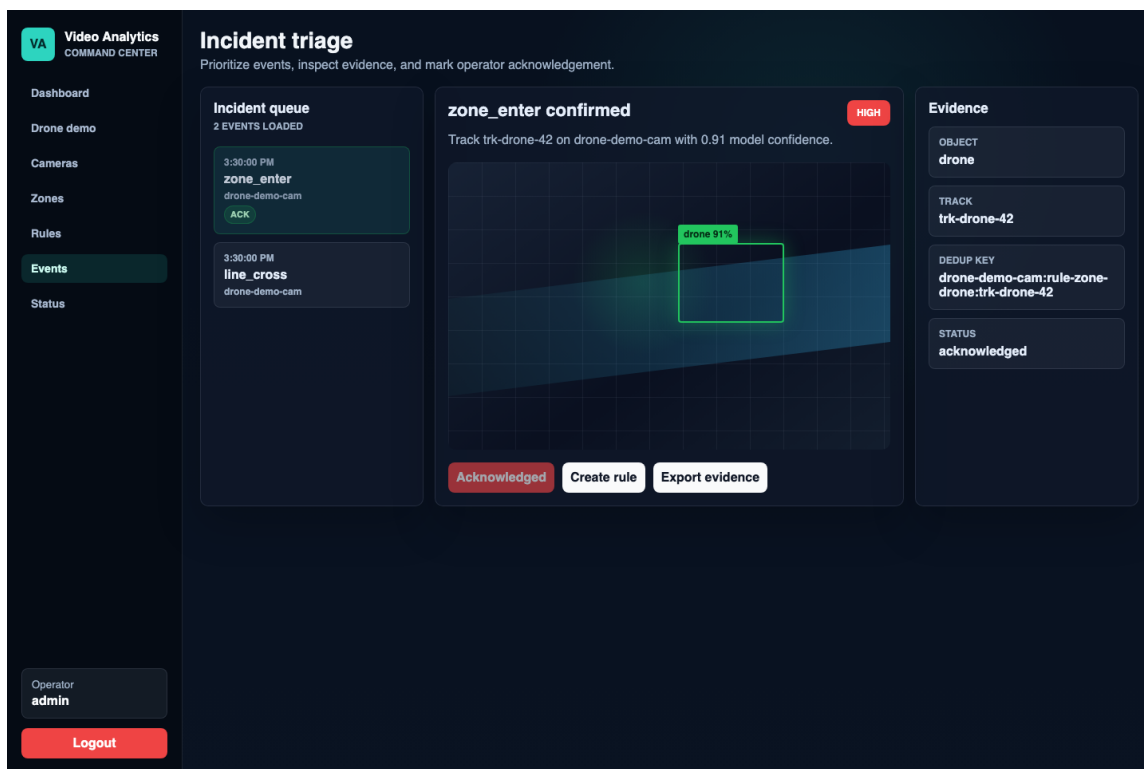


Рисунок 3.3 - Журнал событий с фильтрами по типу правила и временному окну

3.3 Результаты интеграционной проверки

Проведенная серия проверок подтвердила корректность интеграционного маршрута синтетически зафиксированного события `zone_enter`:

событие → исходящее сообщение → брокер → HTTP-получатель. (1)

Для правил `zone_enter`, `line_cross` и `dwell_time` в текущем приемочном прогоне проверялся не полный путь от видеокадра, а прохождение синтетического события через таблицу исходящих сообщений и интеграционный контур. Демонстрационный БПЛА-сценарий отдельно показывает связность видеоконтура от модели и видеофайла до траекторий, событий и операторского интерфейса.

Количественные значения в данном разделе взяты из JSON-артефакта RUN-2026-05-13-A1 и файла `results.csv` обучения детектора. Скриншоты интерфейса и БПЛА-кадры используются как демонстрационные подтверждения связности, а не как источник расчетных метрик.

Таблица 3.1 - Ключевые результаты апробации

Проверка	Результат	Краткий вывод
Интеграционный сценарий	пройден	Синтетическое событие опубликовано и доставлено
zone_enter		
line_cross	синт. событие	Корректный проход через таблицу исходящих сообщений и интеграционный контур
dwell_time	синт. событие	Корректный проход через контур повторной HTTP-доставки
Отказ и восстановление RabbitMQ	пройдено	Потеря не зафиксировано, накопленная очередь исходящих сообщений обработана после восстановления
Повторная HTTP-доставка	пройдена	Повторные попытки отрабатывают, доставка завершается успешно
JWT/RBAC-сценарии	пройдены	Ограничения ролей соблюдаются согласно политике доступа

3.4 Производительность и эксплуатационный смысл метрик

По данным базового приемочного CPU-прогона задержка функциональной доставки составила 4.605 с. Это одиночное измерение интеграционной доставки подтверждает работоспособность контура и задает отправную точку для дальнейшей калибровки на многокамерном стенде; p95-оценка полного пути от объекта к событию требует отдельной длительной серии измерений.

Декомпозиция этапов конвейера схематично показана на рисунке 3.4. Рисунок задает методику последующего профилирования и не является результатом отдельного измерения вклада каждого этапа в текущем приемочном прогоне.

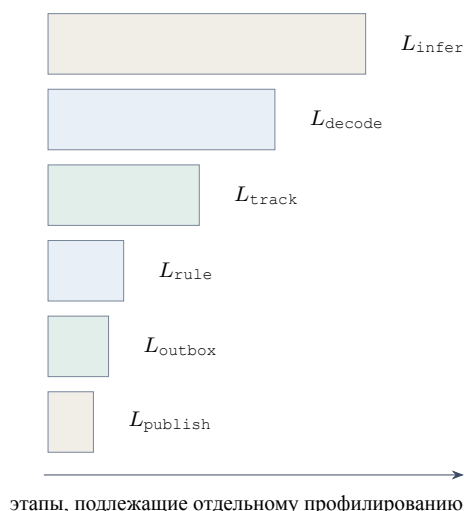


Рисунок 3.4 - Схема декомпозиции задержки по этапам конвейера

3.5 Проверка надежности в отказных режимах

Выполнены сценарии с временной недоступностью RabbitMQ и HTTP-получателя. В обоих случаях подтверждена корректная модель поведения:

- 1) событие не теряется при отказе внешнего компонента;
- 2) запись сохраняется в таблице исходящих сообщений и повторно отправляется;
- 3) после восстановления внешнего контура доставка завершается успешно;
- 4) диагностическая информация отражается в метриках и журналах.

Переход сообщения в очередь недоставленных сообщений при исчерпании повторов проверен отдельным тестом логики фонового обработчика; длительный стендовый прогон до исчерпания повторов в RUN-2026-05-13-A1 не выполнялся.

3.6 Ограничения текущей апробации

Текущая серия испытаний подтверждает архитектурную корректность, но имеет ограничения:

- ограниченный набор сцен и камер;
- отсутствие длительной многокамерной серии;
- отсутствие стендового измерения длительного отказа HTTP-получателя до перехода в очередь недоставленных сообщений;

- отсутствие межкамерной повторной идентификации (ReID);
- необходимость расширения сценариев для оценки редких классов объектов.

3.7 Практические рекомендации

- 1) Начинать внедрение с ограниченного числа камер и фиксированного набора правил.
- 2) Приемку вести по сквозным метрикам (object->event, доставка, backlog), а не только по качеству детектора.
- 3) Сразу включать отказные тесты RabbitMQ и HTTP-получателя в регулярный регламент.
- 4) Настраивать предупреждения на рост числа повторов, сообщений в очереди недоставленных и отброшенных кадров.
- 5) Поддерживать единый протокол прогонов и архив метрик для сравнения релизов.

3.8 Приоритетные направления развития

Для промышленного развития приоритетны следующие шаги:

- расширение классов объектов и дообучение моделей на предметных данных;
- серия длительных многокамерных прогонов с аппаратным ускорением;
- добавление межкамерной корреляции и ReID;
- усиление отказоустойчивости инфраструктуры (кластеризация критичных сервисов);
- оптимизация горячего пути через ускоренные профили исполнения.

ЗАКЛЮЧЕНИЕ

В отчете зафиксированы основные положения магистерского проекта по платформе видеоаналитики реального времени для систем охранного видеонаблюдения.

Сформулирована прикладная постановка задачи: обеспечить контур «обнаружение - сопровождение - событийный анализ - надежная доставка» с контролируемой задержкой и эксплуатационной воспроизводимостью.

В качестве базовой технической линии подтверждена состоятельность связки:

- 1) YOLO-подход для детекции;
- 2) DeepSORT-подход для сопровождения;
- 3) правилый событийный движок для интерпретируемых тревог;
- 4) транзакционная таблица исходящих сообщений, RabbitMQ и повторы HTTP-доставки для надежной доставки.

Результаты апробации подтвердили работоспособность интеграционного контура доставки, корректность RBAC/JWT-политики, устойчивость доставки в базовых отказных сценариях и связность демонстрационного БПЛА-видеосценария. Это позволяет считать цель работы достигнутой на уровне исследовательско-экспериментального стенда: разработана, реализована и апробирована платформа видеоаналитики реального времени для мониторинга охраняемых зон и событийного анализа. Количественные выводы при этом ограничены фактически выполненным приемочным контуром и не подменяют промышленную p95-оценку полного видеоконтура; проверка очереди недоставленных сообщений зафиксирована отдельно как проверка логики фонового обработчика.

Дальнейшее развитие связано с масштабированием на многокамерные профили с аппаратным ускорением, расширением набора классов и углублением межкамерной аналитики. Таким образом, получена воспроизводимая основа для перехода к промышленной системе мониторинга охраняемых зон и событийного анализа.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. You Only Look Once: Unified, Real-Time Object Detection / J. Redmon [и др.] // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. — 2016. — С. 779—788. — DOI: 10.1109/CVPR.2016.91.
2. *Wojke N., Bewley A., Paulus D.* Simple Online and Realtime Tracking with a Deep Association Metric // 2017 IEEE International Conference on Image Processing (ICIP). — 2017. — С. 3645—3649. — DOI: 10.1109/ICIP.2017.8296962.
3. *microservices.io*. Pattern: Transactional Outbox. — 2026. — URL: <https://microservices.io/patterns/data/transactional-outbox.html> (дата обр. 30.04.2026).
4. *RabbitMQ Team*. RabbitMQ Documentation. — 2026. — URL: <https://www.rabbitmq.com/documentation.html> (дата обр. 30.04.2026).
5. *FastAPI*. FastAPI Documentation. — 2026. — URL: <https://fastapi.tiangolo.com/> (дата обр. 30.04.2026).
6. *PostgreSQL Global Development Group*. PostgreSQL Documentation. — 2026. — URL: <https://www.postgresql.org/docs/> (дата обр. 30.04.2026).
7. *Docker*. Docker Documentation. — 2026. — URL: <https://docs.docker.com/> (дата обр. 30.04.2026).
8. *Prometheus Authors*. Prometheus Documentation. — 2026. — URL: <https://prometheus.io/docs/> (дата обр. 30.04.2026).
9. *Grafana Labs*. Grafana Documentation. — 2026. — URL: <https://grafana.com/docs/> (дата обр. 30.04.2026).